The templates used in route attributes are standard route templates, the same as you used in chapter 6. You can use literal segments, and you're free to define route parameters that will extract values from the URL, as shown by the `SetCarSpeed` method in listing 20.5. That method defines two route templates, both of which define a route parameter, `{speed}`.

**TIP** I've used multiple `[Route]` attributes on each action in this example, but it's best practice to expose your action at a single URL. This will make your API easier to understand and for other applications to consume.

As in all parts of ASP.NET Core, route parameters represent a segment of the URL that can vary. As with minimal APIs, and Razor Pages, the route parameters in your `RouteAttribute` templates can

- Be optional
- Have default values
- Use route constraints

For example, you could update the `SetCarSpeed` method in the previous listing to constrain `{speed}` to an integer and to default to `20` like so:

```
[Route("car/speed/{speed=20:int}")]
[Route("set-speed/{speed=20:int}")]
public IActionResult SetCarSpeed(int speed)
```

**NOTE** As discussed in chapter 6, don't use route constraints for validation. For example, if you call the preceding `"set-speed/{speed=20:int}"` route with an invalid value for speed, `/set-speed/oops`, you will get a `404 Not Found` response, as